

LLM-агенты для разработки GPU-ядер в незнакомом фреймворке и незнакомом числовом формате: CuTe DSL и FP8

AIRI Summer 2026

31 июля 2026 г.

1 Постановка задачи

Проект исследует два аспекта построения LLM-агента, который разрабатывает и оптимизирует GPU-ядра внутри фреймворка, плохо знакомого модели.

Первое направление — освоение нового фреймворка прямо в процессе работы агента. Современные модели неплохо справляются с Triton, но испытывают трудности с менее распространёнными абстракциями, такими как CuTe. Чтобы понять, чем именно можно компенсировать этот пробел, нужен харнесс, который подаёт модели документацию, примеры кода, результаты компиляции, тесты и данные профилирования, — и на нём уже можно сравнивать стратегии извлечения информации и обратной связи. Ожидаемый результат — анализ инструментов, позволяющих LLM освоить новый GPU-фреймворк *без дообучения*.

Второе направление — незнакомые числовые форматы на примере FP8. Переход с FP16 на FP8 требует учёта специфики кодирования и специализированных инструкций, поэтому здесь нужен харнесс для генерации FP8-кода и устойчивый замкнутый цикл разработки. Оба направления мы измеряем одними и теми же метриками: корректность, ускорение, число итераций агента и стоимость в токенах.

2 Инфраструктура эксперимента

Работу мы начали с того, что развернули модель у себя. Мы задеплоили Qwen3.6-35B-A3B в квантизации Q8_0 на сервере с 2×V100 и запустили её с пайплайн-параллелизмом (PP = 2), после чего открыли доступ через OpenAI-совместимый эндпоинт. Модель довольно маленькая и сама по себе писать на CuTe DSL не умеет — именно поэтому она удобна как объект исследования: всё, что она сможет, будет заслугой харнесса, а не предобучения.

Дальше нужно было чем-то измерять результат. Кандидатные ядра мы выполняем удалённо на NVIDIA B300, причём оценщик, а не агент, владеет генерацией входов, эталоном на Torch, проверкой допусков, замером времени и вердиктом PASS; агент правит только файл кандидата и не может подменить проверку. Чтобы замеры были стабильными, мы делаем несколько разминочных прогонов, затем несколько основных и усредняем; эталонное ядро тайминится отдельно и служит знаменателем ускорения, но модели никогда не показывается.

Наконец, нам понадобились сами задачи — на CuTe DSL и в FP8. Готовых бенчмарков такого рода не существует, поэтому мы адаптировали задачи из KernelBench: переиспользовали их формат, но написали собственные эталонные решения, привели входы и эпилоги к FP8-семантике и зафиксировали для каждой задачи свои пороги численного расхождения. В итоге получилось девять задач первого и второго уровней.

Справочные материалы по CuTe DSL мы разбили на кумулятивные уровни, не зависящие от конкретной задачи: BARE (только задача), API (CuTe DSL API и алгебра layout'ов), ARCHITECTURE (Blackwell, TMA, TMEM, численность FP8/FP4), EXAMPLES (нейтральные фрагменты кода) и ERROR HINTS (типовые классы ошибок).

3 Итерация 1 — харнесс на OpenCode

Первый харнесс устроен максимально просто: на задачу запускается один агент OpenCode, который читает условие, starter и установленный справочник по CuTe, может вызывать команды харнесса `check` и `run`, чтобы проверить себя, и оставляет финального кандидата. Без документации этот агент не решает ровно ни одной задачи, так что нижняя граница здесь очевидна. С полным стеком документации картина меняется качественно (таблица 1): все девять задач доходят до PASS за один независимый запуск, то есть при достаточно полном справочнике модель способна писать корректные FP8-ядра на CuTe DSL.

Таблица 1: Агент OpenCode, один запуск на задачу, Qwen3.6-35B-A3B (Q8_0). Время в мс.

Задача	Статус	Эталон	Агент	Ускор.	Вход	Кэш вход	Выход	Сек.
level1_01 square_matmul	PASS	0.1439	0.1419	1.014×	23,572	274,374	5,178	162.5
level1_40 layer_norm	PASS	8.3431	36.6102	0.228×	18,162	128,542	2,260	90.6
level1_72 conv_transpose3d	PASS	—	—	—	25,982	313,984	5,468	163.7
level2_09 matmul_sub_mul_relu	PASS	0.3849	0.1521	2.530×	22,832	190,529	4,879	147.5
level2_12 gemm_mul_leaky_relu	PASS	0.3870	0.1531	2.529×	23,884	112,455	4,571	130.4
level2_14 gemm_div_sum_scale	PASS	0.3632	0.1321	2.749×	25,058	391,956	7,958	272.0
level2_40 matmul_scale_resid	PASS	0.3859	0.1538	2.509×	23,401	121,455	4,536	134.4
level2_63 gemm_relu_divide	PASS	0.3849	0.1508	2.552×	22,517	142,166	4,512	132.3
level2_76 gemm_add_relu	PASS	0.1242	0.1468	0.846×	28,591	161,684	4,728	150.5

4 Итерация 2 — kernel-evo, режим evolve, абляция уровней

Убедившись, что задача в принципе решается, мы перенесли харнесс в целевой KERNEL-EVO и поставили следующий вопрос: какой именно уровень документации отвечает за этот результат. Для этого мы запустили режим evolve с одним островом и двумя мутациями на серию на трёх задачах — `level1_01_square_matrix_multiply`, `level1_40_layer_norm_fp8` и `level1_02_vector_scale_fp4`. Каждый уровень документации получает три серии по две попытки, то есть шесть попыток суммарно, и считается решённым для серии, если хотя бы одна попытка дошла до PASS.

Таблица 2: Режим evolve, абляция уровней документации: три задачи × две мутации на уровень.

Уровень	Успешные попытки	Решённые серии	Выход/попытку
Bare	0/6	0/3	367
API	0/6	0/3	229
Architecture	0/6	0/3	229
Examples	2/6	2/3	287
Error hints	5/6	3/3	382

Результат (таблица 2) оказался устроен монотонно, но не так, как можно было бы ожидать от объёма поданного текста. Уровни API и ARCHITECTURE сами по себе не решают ничего: модель получает несколько десятков тысяч токенов связного описания API и архитектуры Blackwell и остаётся ровно там же, где была на BARE. Перелом наступает на EXAMPLES — две решённые серии из трёх — и закрепляется на ERROR HINTS, где решены все три серии при пяти успешных попытках из шести. Иными словами, эффект несут конкретные артефакты: проработанные примеры кода и описания типовых ошибок, а не прозаические главы про API. Косвенно это подтверждает и длина ответа: на уровнях, где модель не понимает, что делать, она пишет вдвое короче (229 токенов на попытку против 382 на ERROR HINTS), то есть быстрее сдаётся, а не дольше ошибается. Это наблюдение и предстоит перепроверить в E1 — уже в агентском режиме и на задачах второго уровня.

5 Запланированные эксперименты (E1–E4)

Дальнейшие эксперименты идут по одной схеме: три задачи второго уровня (`level2_12`, `level2_14`, `level2_40`), шесть ходов на запуск, одна репликация на ячейку и одна реплика модели, причём в каждом эксперименте меняется ровно один параметр относительно лучшего уровня документации по итогам E1.

Таблица 3: Сетка экспериментов и результаты. Все ячейки заполняются по итогам запусков.

Эксп.	Вопрос	Плечо	Pass	Ускор.	Вход	Выход
E1	Какое знание важно	уровни × 3 задачи (12 зап.)	TBD	TBD	TBD	TBD
E2	Стратегия извлечения	доставка <code>files</code> vs <code>prompt</code> (3 зап.)	TBD	TBD	TBD	TBD
E3	Обратная связь: агент-критик	<code>feedback.critic</code> (3 зап.)	TBD	TBD	TBD	TBD
E4	Обратная связь: профайлер	<code>profiling.enabled</code> (3 зап.)	TBD	TBD	TBD	TBD

E1 повторяет лестницу уровней в агентском режиме и фиксирует опорную конфигурацию для всего остального. E2 сравнивает два механизма доставки одного и того же корпуса: файлы уровня, материализованные в рабочей директории и открываемые агентом по необходимости, против полного бандла,

вставляемого в промпт на каждом ходу; интересна здесь прежде всего стоимость входных токенов на одно решение. E3 добавляет между ходами один вызов агента-критика, работающего только на чтение: он получает кандидата, диагностику с V300 и тот же уровень документации и возвращает короткие подсказки, а его токены учитываются отдельно от токенов автора. E4 возвращает следующему ходу компактную сводку по ядрам с V300; поскольку профиль появляется только после успешного запуска, ожидаемый эффект здесь — на ускорение среди прошедших запусков, а не на pass rate. К каждому эксперименту прилагается таксономия ошибок (import/API, компиляция, layout и формы, численное расхождение, таймаут): **TBD**.